Angular Pipes: Transforming Data in Templates

---

1. Introduction to Pipes

The Problem

In web applications, data often doesn't arrive from the server in a user-friendly format. A date might look like `2024-03-15T10:30:00.000Z`, a price like `1999.5`, or text might be in the wrong case.

Without Pipes (The Messy Way):

```
// In your component
export class ProductComponent {
  price = 1999.5;
  formattedPrice = '$' + this.price.toFixed(2); // Doing logic in component
}
```

```HTML
<!— In your template —>
<p>Total: {{ formattedPrice }}</p>
```

This approach clutters your component with presentation logic.

The Solution: Pipes

Pipes are simple functions you can use in template expressions to transform data directly in your HTML. They are **declarative**, **reusable**, and **keep your component clean**.

With Pipes (The Angular Way):

```HTML
<!— Data transformation happens right in the template! —>
<p>Total: {{ price | currency:'USD' }}</p>
<p>Created: {{ createdAt | date:'fullDate' }}</p>
```

Think of pipes like filters in Unix/Linux command lines—they take an input, transform it, and output the result.

---

2. Syntax of Pipes

Basic Syntax

```
{{ value | pipeName }}
```

Pipes with Parameters

Some pipes accept parameters to customize their behavior. Parameters are added using a colon (`:`) and can be chained.

```
{{ value | pipeName:parameter1:parameter2 }}
```

Chaining Pipes

You can apply multiple transformations sequentially using the pipe symbol (`|`) multiple times.

```
{{ value | pipeA | pipeB }}
```

Execution order: Left to right. `pipeA` transforms `value`, then `pipeB` transforms the result of `pipeA`.

Example Walkthrough

```html
<!— Original data —>
<p>{{ 5.678 }}</p>  <!— Output: 5.678 —>

<!— Using number pipe to format —>
<p>{{ 5.678 | number:'1.2-2' }}</p>  <!— Output: 5.68 —>

<!— Chaining: uppercase then number (doesn't make sense, but shows syntax) —>
<!— Usually you chain date → uppercase, etc. —>
```

---

3. Built-in Pipes (The Essentials)

Angular comes with a rich set of built-in pipes. Let's explore the most commonly used ones.

3.1 `DatePipe` - Formatting Dates

Formats a date value according to locale rules.

Syntax: `{{ dateValue | date:format:timezone:locale }}`

Common Formats:

```
// In component
today: Date = new Date(); // e.g., 2024-03-15T10:30:00
```

```html
<!— Default format: 'MMM d, y' (mediumDate) —>
<p>{{ today | date }}</p>  <!— Output: Mar 15, 2024 —>

<!— Predefined formats —>
<p>{{ today | date:'short' }}</p>    <!— 3/15/24, 10:30 AM —>
<p>{{ today | date:'medium' }}</p>   <!— Mar 15, 2024, 10:30:00 AM —>
<p>{{ today | date:'long' }}</p>     <!— March 15, 2024 at 10:30:00 AM GMT+5:30 —>

<!— Custom formats —>
<p>{{ today | date:'dd/MM/yyyy' }}</p>    <!— 15/03/2024 —>
<p>{{ today | date:'EEEE, MMMM d' }}</p>  <!— Friday, March 15 —>
```

3.2 `UpperCasePipe` & `LowerCasePipe` - Changing Text Case

Transform text to uppercase or lowercase.

```
message: string = 'Hello Angular World';
```

```html
<p>{{ message | uppercase }}</p>  <!— HELLO ANGULAR WORLD —>
<p>{{ message | lowercase }}</p>  <!— hello angular world —>
<p>{{ 'Static Text' | uppercase }}</p>  <!— STATIC TEXT —>
```

3.3 `TitleCasePipe` - Capitalizing Words

Capitalizes the first letter of each word.

```
title: string = 'the lord of the rings';
```

```html
<p>{{ title | titlecase }}</p>  <!— The Lord Of The Rings —>
```

3.4 `CurrencyPipe` - Formatting Money

Formats a number as currency.

Syntax: `{{ value | currency:currencyCode:display:digitsInfo:locale }}`

```
price: number = 1299.95;
```

```html
<!— Default: USD, symbol —>
<p>{{ price | currency }}</p>  <!— $1,299.95 —>

<!— Different currency —>
<p>{{ price | currency:'EUR' }}</p>  <!— €1,299.95 —>
<p>{{ price | currency:'JPY' }}</p>  <!— ¥1,300 (JPY has no decimals) —>

<!— Control display —>
<p>{{ price | currency:'INR':'symbol-narrow' }}</p>  <!— ₹1,299.95 —>
<p>{{ price | currency:'GBP':'code' }}</p>  <!— GBP1,299.95 —>
```

3.5 `DecimalPipe` (aka `number`) - Formatting Numbers

Formats a number with decimal points and thousand separators.

Syntax: `{{ value | number:'digitsInfo' }}`
DigitsInfo format: `{minIntegerDigits}.{minFractionDigits}-{maxFractionDigits}`

```
num1: number = 3.14159265;
num2: number = 0.45;
largeNum: number = 9876543.21;
```

```html
<p>{{ num1 | number:'1.2-2' }}</p>   <!— 3.14 (min 2, max 2 fractions) —>
<p>{{ num1 | number:'1.2-5' }}</p>   <!— 3.14159 (min 2, max 5 fractions) —>
<p>{{ num2 | number:'1.2-2' }}</p>   <!— 0.45 —>
<p>{{ num2 | number:'3.2-2' }}</p>   <!— 000.45 (min 3 integers) —>
<p>{{ largeNum | number }}</p>        <!— 9,876,543.21 (default) —>
```

3.6 `PercentPipe` - Formatting Percentages

Formats a number as a percentage.

```
progress: number = 0.85;
```

```html
<p>{{ progress | percent }}</p>           <!— 85% —>
<p>{{ progress | percent:'1.2-2' }}</p>   <!— 85.00% —>
```

3.7 `SlicePipe` - Extracting Substrings/Arrays

Creates a new array or string containing a subset of the elements.

Syntax: `{{ value | slice:start:end }}` (end is exclusive)

```
text: string = 'abcdefghij';
items: string[] = ['Apple', 'Banana', 'Cherry', 'Date', 'Elderberry'];
```

```HTML
<!— Strings —>
<p>{{ text | slice:0:3 }}</p>  <!— abc —>
<p>{{ text | slice:3:6 }}</p>  <!— def —>
<p>{{ text | slice:-3 }}</p>   <!— hij (negative starts from end) —>

<!— Arrays —>
<ul>
  <li *ngFor="let item of items | slice:1:4">
    {{ item }}  <!— Banana, Cherry, Date —>
  </li>
</ul>
```

3.8 `JsonPipe` - Debugging Objects

Converts a value to its JSON-string representation. Great for debugging.

```
user = { name: 'John', age: 30, hobbies: ['reading', 'coding'] };
```

```HTML
<pre>{{ user | json }}</pre>
<!— Output:
{
  "name": "John",
  "age": 30,
  "hobbies": ["reading", "coding"]
}
—>
```

3.9 `KeyValuePipe` - Iterating Over Objects

Transforms an object into an array of key-value pairs for iteration with `*ngFor`.

```
person = { firstName: 'Jane', lastName: 'Doe', age: 28 };
```

```html
<div *ngFor="let item of person | keyvalue">
  {{ item.key }}: {{ item.value }}
</div>
<!— Output:
firstName: Jane
lastName: Doe
age: 28
—>
```

3.10 `AsyncPipe` - Handling Observables/Promises

Crucial for reactive programming! Subscribes to an Observable or Promise and returns the latest emitted value. It also handles unsubscription automatically, preventing memory leaks.

```
// In component
import { Observable, of } from 'rxjs';

// Simulating an async data stream
time$: Observable<string> = new Observable(observer => {
  setInterval(() => {
    observer.next(new Date().toLocaleTimeString());
  }, 1000);
});

// Or a simple promise
dataPromise: Promise<string> = Promise.resolve('Async Data Loaded');
```

```html
<!— Without async pipe (wrong approach - won't update) —>
<p>{{ time$ }}</p>  <!— [object Object] - not what we want! —>

<!— WITH async pipe - automatically subscribes and updates —>
<p>Current time: {{ time$ | async }}</p>  <!— Updates every second —>
<p>Promise data: {{ dataPromise | async }}</p>  <!— "Async Data Loaded" —>

<!— Common pattern with *ngIf and async —>
<div *ngIf="(userData$ | async) as user">
  Hello, {{ user.name }}!
</div>
```

---

4. Understanding Pipe Categories

Pure Pipes (Default)

- **Pure** means the pipe is triggered only when the **primitive value** or **object reference** changes.

- Most built-in pipes are pure.
- **Performance:** Excellent. Angular ignores changes inside objects/arrays.

Impure Pipes

- An **impure** pipe is called during every component change detection cycle (e.g., every keystroke, mouse move, timer tick).
- **Performance:** Can be very expensive. Use sparingly and only when necessary.
- **Built-in impure pipes:** `async`, `json` (in some contexts), `keyvalue` (creates new array reference).

---

5. Custom Pipes

When built-in pipes aren't enough, you can create your own!

Why Create a Custom Pipe?

- **Reusability:** Encapsulate transformation logic once, use everywhere.
- **Separation of Concerns:** Keep templates clean and components focused on business logic.
- **Specific Domain Logic:** e.g., "Truncate text after 100 characters", "Filter a list based on a search term", "Format a phone number".

Step-by-Step: Creating a Custom Pipe

Step 1: Generate the Pipe

Using Angular CLI (recommended):

```shell
ng generate pipe pipes/truncate
# or shorthand
ng g p pipes/truncate
```

This creates `truncate.pipe.ts` and updates the module (or standalone component imports).

Step 2: Understand the Generated Code

```typescript
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'truncate',  // This is what you'll use in templates
  standalone: true   // or false if using NgModules
})
export class TruncatePipe implements PipeTransform {

  transform(value: unknown, ...args: unknown[]): unknown {
    return null;
  }

}
```

Step 3: Implement the `transform` Method

The `transform` method is where the magic happens.

Method Signature:

```
transform(value: any, ...args: any[]): any
```

- `value`: The input being piped (left side of `|`).
- `args`: Optional parameters passed to the pipe (right side of `:`).

Example 1: Text Truncation Pipe

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'truncate',
  standalone: true
})
export class TruncatePipe implements PipeTransform {

  transform(value: string, limit: number = 20, trail: string = '...'): string {
    if (!value) return '';
    if (value.length ≤ limit) return value;

    return value.substring(0, limit) + trail;
  }

}
```

Usage:

```
<p>{{ longText | truncate }}</p>  <!— Default 20 chars + ... —>
<p>{{ longText | truncate:10 }}</p>  <!— 10 chars + ... —>
<p>{{ longText | truncate:15:'---' }}</p>  <!— 15 chars + --- —>
```

Example 2: Phone Number Formatter

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'phone',
  standalone: true
})
export class PhonePipe implements PipeTransform {

  transform(phoneNumber: string | number): string {
    if (!phoneNumber) return '';

    // Remove all non-numeric characters
    const cleaned = String(phoneNumber).replace(/\D/g, '');

    // Check if it's a 10-digit US number
    if (cleaned.length === 10) {
      return `(${cleaned.slice(0,3)}) ${cleaned.slice(3,6)}-${cleaned.slice(6)}`;
    } else if (cleaned.length === 11 && cleaned[0] === '1') {
      // International with country code
      return `+1 (${cleaned.slice(1,4)}) ${cleaned.slice(4,7)}-${cleaned.slice(7)}`;
    }

    // Return original if not matching expected format
    return String(phoneNumber);
  }

}
```

Usage:

```html
<p>{{ 1234567890 | phone }}</p>  <!— (123) 456-7890 —>
<p>{{ '1234567890' | phone }}</p>  <!— (123) 456-7890 —>
<p>{{ '11234567890' | phone }}</p>  <!— +1 (123) 456-7890 —>
```

Example 3: Filter Array (Impure - Use with Caution!)

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'filter',
  pure: false,  // Impure pipe - re-evaluates on every change
  standalone: true
})
export class FilterPipe implements PipeTransform {

  transform(items: any[], searchTerm: string, property: string = 'name'): any[] {
    if (!items || !searchTerm) return items;

    return items.filter(item =>
      item[property].toLowerCase().includes(searchTerm.toLowerCase())
    );
  }

}
```

Usage:

```HTML
<input [(ngModel)]="searchText" placeholder="Search ... ">
<ul>
  <li *ngFor="let product of products | filter:searchText:'name'">
    {{ product.name }}
  </li>
</ul>
```

⚠️ **WARNING:** This impure pipe runs on every change detection cycle, which can be very slow for large lists. A better approach is to handle filtering in the component using signals or observables.

---

6. Pipe Best Practices

✅ DO:

- Use pipes for simple, **stateless** transformations.
- Leverage built-in pipes before creating custom ones.
- Make custom pipes **pure** whenever possible (the default).
- Use the `async` pipe for observables to manage subscriptions automatically.
- Place custom pipes in a `pipes` folder for organization.

❌ DON'T:

- Use pipes for complex data manipulation that belongs in services or components.
- Create impure pipes for filtering/sorting large lists (use component logic instead).

- Forget to register/import your custom pipes.
- Use pipes that cause **side effects** (modifying data, making HTTP calls, changing component state). Pipes should be idempotent.

Pure vs Impure Decision Guide

Scenario	Recommended Pipe Type
String formatting (date, currency, case)	Pure
Object transformation based on fixed logic	Pure
Filtering/sorting arrays	Component Logic (not pipe)
Async data subscription	Pure (async pipe handles subscription)
Debugging with changing objects	Impure (but use `json` pipe sparingly)

---

7. Complete Example: Putting It All Together

Scenario: An e-commerce product card displaying various transformed data.

Component:

```
// product-card.component.ts
import { Component, Input } from '@angular/core';
import { CommonModule } from '@angular/common';
import { TruncatePipe } from '../pipes/truncate.pipe';
import { PhonePipe } from '../pipes/phone.pipe';

@Component({
  selector: 'app-product-card',
  standalone: true,
  imports: [CommonModule, TruncatePipe, PhonePipe],
  template: `
    <div class="product-card">
      <h2>{{ product.name | uppercase }}</h2>
      <p class="price">{{ product.price | currency:'USD':'symbol':'1.2-2' }}</p>

      <p class="description">
        {{ product.description | truncate:50 }}
      </p>

      <div class="meta">
        <span>Added: {{ product.addedDate | date:'mediumDate' }}</span>
        <span>SKU: {{ product.sku | lowercase }}</span>
      </div>

      <div class="contact">
        Seller: {{ product.sellerName | titlecase }}
        <br>
        Phone: {{ product.sellerPhone | phone }}
      </div>

      <!— Async data example —>
      <div *ngIf="stockStatus$ | async as status" class="stock">
        Stock: {{ status }}
      </div>
    </div>
  `
})
export class ProductCardComponent {
  @Input() product: any;

  // Simulating async stock status
  stockStatus$ = new Observable<string>(observer => {
    setTimeout(() => observer.next('In Stock'), 1000);
  });
}
```

8. Summary Cheat Sheet

Pipe	Purpose	Example
`date`	Format dates	`{{ today | date:'fullDate' }}`
`uppercase` / `lowercase`	Change text case	`{{ 'hello' | uppercase }}`
`currency`	Format money	`{{ 99.99 | currency:'EUR' }}`
`number`	Format numbers	`{{ 3.14159 | number:'1.2-2' }}`
`percent`	Format percentages	`{{ 0.25 | percent }}`
`slice`	Extract portion	`{{ 'abcdef' | slice:0:3 }}`
`json`	Debug objects	`{{ user | json }}`
`keyvalue`	Iterate objects	`*ngFor="let item of obj | keyvalue"`
`async`	Unwrap observables	`{{ data$ | async }}`
Custom	Your logic	`{{ text | customPipe:param }}`